File Operations, Command-Line Arguments, Exception Handling

~ G. Anushiya Rachel

Assistant Professor, CSE

Shiv Nadar University Chennai

File Operations

- Assume that you have a file named, $textfile.txt$
- Open the file using ***open***:
  - Syntax: $fh = open(filename, mode)$
  - Modes:
    - r – Read (default mode) – Returns an error if the file does not exist
    - w – Write – Creates the file if it does not exist
    - a – Append – Creates the file if it does not exist
    - x – Create – Returns an error of the file exists
    - $r+, w+, a+$ - Reading and writing/appending

  >>> fh = open("textfile.txt", "r")
- Close the file using ***close***:

  >>> fh.close()

File Operations

- Use ***with open*** to open files, to avoid having to close files:
  - $with\ open("text.txt","r")\ as\ fh:$

    $content = fh.read(\quad)$

    $print(content)$
- Read the contents of a file using ***read, readline, readlines***:
  - $fh.read(\quad)$ - Reads the entire content of a file (from the position of the file pointer/handle) and returns a string
  - $fh.readline(\quad)$ - Reads one line from the file, from the current position
  - $fh.readlines(\quad)$ - Reads all lines from the current position to the end of the file and returns a list containing each line

File Operations

- Move to a desired position in a file using ***seek***:
  - $fh.seek(offset, reference_point)$
    - Reference point can take one of three values:
      - 0 – beginning of the file
      - 1 – current position in the file
      - 2 – end of the file
- Find current position in a file using ***tell***:
  - $fh.tell(\quad)$
- Write using ***write, writelines***:
  - $fh.write(\quad)$ - To write a string
  - $fh.writelines(\quad)$ - To write a list of strings

Note: Offset can be non-zero only when the reference point is 0, when reading text files.

Handling Directories/Folders

- Use the ***os*** module.
- $os.getcwd(\quad)$ – Get the path to the current working directory
- $os.chdir(directory_name)$ – Change to the specified directory
- $os.listdir(directory_name)$ – List the contents of the specified directory
- $os.mkdir(directory_name)$ – Create a new directory
- $os.rename(old_name, new_name)$ – Rename the specified directory
- $os.remove(filename)$ – Delete a file
- $os.rmdir(directory_name)$ – Delete a directory
- $os.walk(directory_name)$ – Returns directory path, directory names, file names

Command-Line Arguments

- Accept command line arguments using the ***sys*** module
- $sys.argv$ – list of command-line arguments
  - $sys.argv[0]$ – Name of the Python program
- To find the number of command-line arguments – $len(sys.argv)$
- To access the ith argument – $sys.argv[i]$

Exception Handling

- Exceptions can be handled using the ***try … except*** block.
- Syntax:

```
try:
    # statements that may cause an exception or error
except:
    # statements to run when an exception occurs
```

- Catch specific exceptions – *except* must be followed by the exception, say *ZeroDivisionError* or *IndexError*, etc.
  - List of built-in exceptions can be obtained by the following statement:

```
print(dir(locals()['__builtins__']))
```

Exception Handling

- With ***else*** block:
  - The *else* block contains code that has to be run if the *try* block runs successfully.
  - Example:

try:

$num = int(input(\quad))$

$d = 1/num$

$except$:

$print(\text{"num must not be equal to zero"})$

$else$:

$ans = 10 * d$

Exception Handling

- With ***finally*** block:
  - *finally* block is executed whether there is an exception or not.
  - Example:

```
try:
        n1 = int(input(  ))
        n2 = int(input(  ))
        n3 = n2/n3
except:
        print("Error")
finally:
        print("Done")
```

Programming Exercises

1. Write a program that takes a text file as an input. Read only the first n lines and print them. Then skip the next k characters and print the remaining characters in the line. Then, skip k lines and print the remaining lines.
2. Write a program that takes a text file as an input argument. Check if the file exists (Look for $FileNotFoundError$). Find the number of words, lines, and characters in this file. Remove punctuations from the file and find the frequency of occurrence of each word.
3. Create a directory and create 3 sub-directories within this using the os module. Create 2 text files inside each subdirectory using file. Explore the use of $os.walk$.

Programming Exercises

4. Create a folder containing a 5 text files. Each text file should contain a list of preferred usernames. Check if each of the user names are valid (contain only alphabets, digits, '_', '~', '.'). If a username is invalid remove the invalid characters from it. Add the domain name, 'snuchennai.edu.in', to all the usernames and create email addresses. Write these email addresses into new files in a different folder.
   - Obtain the input and output folders as command-line arguments. Exit the program if the number of arguments is incorrect.
   - If the input folder does not exist or if it does not contain 5 files, print an error message and exit the program.
   - If the output folder does not exist, create one. If the output folder exists and contains some files, ask the user if the contents of the folder can be deleted. If the user responds with a 'yes', delete the contents and proceed. If the user responds with a 'no', exit the program.